

Omi Platform

Technical Design Document

Version: 0.1.0

Date: June 2026

Status: Active Development

Table of Contents

1. Executive Summary
2. Architectural Principles
3. System Components
4. API Contracts
5. Task Routing Flow
6. Task Execution Lifecycle
7. Architecture Diagrams
8. Security & Compliance
9. Scaling & Deployment
10. Extension Points
11. Key Metrics & SLOs

1. Executive Summary

Omi Platform is a **multi-agent orchestration system** that enables seamless integration and management of distributed AI agents. It provides:

- **Single entry point** for all clients to submit tasks to a pool of agents
- **Intelligent routing** that matches tasks to the best available agent based on capability, keywords, or LLM decision
- **Task chaining** where outputs of one task feed into the next for multi-step workflows
- **Collaboration mode** to solicit simultaneous responses from multiple agents
- **Built-in observability** for cost tracking, token usage, and performance metrics
- **Zero external API dependency** — agents run independently, platform only coordinates

2. Architectural Principles

2.1 Core Principles

Principle	Description
Agent Autonomy	Agents are self-contained services with their own lifecycle. The platform does not manage agent internals.
Stateless Routing	Task routing is deterministic: explicit → capability hint → keyword match → LLM fallback.
Heartbeat-driven Status	Agents must heartbeat every 30s or they're marked offline. No polling from platform.
Request-scoped Context	Task context and metadata flow through the chain; no persistent session state on platform.
Cost & Token Transparency	Every task result tracks tokens, cost, and duration for billing and optimization.
Security by Default	JWT authentication, CORS restrictions, trusted host validation, security headers on all responses.

2.2 Design Trade-offs

Choice	Why	Trade-off
Heartbeat over WebSocket	Simple, stateless monitoring for distributed agents	Agents must implement heartbeat loop; slower failure detection
Keyword + LLM routing	Fast fallback without ML infrastructure	Less optimal for novel task types; LLM routing adds ~15ms latency
In-process SQLite	Easier deployment in dev/single-server scenarios	Not suitable for large-scale multi-instance deployments
Task chain vs. DAG	Simple, easy to reason about sequential dependencies	No support for parallel task execution or complex workflows

3. System Components

3.1 Component Overview

Figure 1: High-Level System Architecture

```
HTMLEOF # Read and embed the first SVG cat /mnt/d/dev/addy_wp/omi/diagrams/01-
architecture.svg >> /mnt/d/dev/addy_wp/omi/TECHNICAL_DESIGN_WITH_DIAGRAMS.html cat
>> /mnt/d/dev/addy_wp/omi/TECHNICAL_DESIGN_WITH_DIAGRAMS.html << 'HTMLEOF'
```

3.2 Core Components

API Gateway (FastAPI)

Single HTTP entry point for all clients. Defines routes: `/agents`, `/tasks`, `/auth`, `/health`, `/ws`.

- Middleware: CORS, trusted host validation, security headers
- Lifespan management: initializes registry, storage, observability on startup
- Accepts task submissions (REST + WebSocket)
- Enforces JWT authentication
- Streams WebSocket results in real-time

Route Manager (Dispatcher)

Intelligent task-to-agent mapping using a 4-level cascade:

1. **Explicit** — `task.agent_id` is set → direct route
2. **Capability Hint** — `task.metadata["capability"]` matches a tag
3. **Keyword Matching** — Analyze message for patterns
4. **LLM Fallback** — Ask Claude Haiku to pick the best agent

Levels 1-3 are fast (< 10ms); level 4 adds ~15ms but is semantically aware.

Registry Store

Maintains source of truth for agent availability and capabilities.

- `register_agent()` — Agent joins (idempotent; re-register = update)
- `deregister_agent()` — Agent leaves
- `heartbeat()` — Agent pings every 30s; updates status
- `list_agents()` — Query by status or capability
- `find_agents_for_task()` — Return online agents matching capabilities

Background process: Health monitor runs every 30s, marks agents OFFLINE if no heartbeat for 90s.

Task Executor

Orchestrates task submission and result persistence.

- Save task to storage (status=PENDING)
- Route task to an agent
- Call agent's `/run` endpoint (300s timeout)
- Persist TaskResult to storage
- Record usage (tokens, cost, duration)

Storage Layer (SQLAlchemy + SQLite)

TaskRecord — Immutable log of every task execution (indexed by session_id, agent_id)

AgentRecord — Agent registry (indexed by name, status, capabilities)

UsageRecord — Cost & performance tracking (indexed by task_id, agent_id)

Authentication (JWT)

- Tokens issued at `/auth/token` after validating username/password
- Token embedded in WebSocket query param: `ws://host:9000/ws/SESSION_ID?token=JWT_TOKEN`
- Tokens expire after 24 hours (configurable)
- In production: integrate with LDAP, OAuth2, etc.

Observability Tracker

Real-time tracking of platform usage across all agents.

- Total tasks executed
- Total tokens consumed (for billing)
- Total cost (USD)
- Average duration per task
- Per-agent breakdowns

Agent SDK (agent_base.py)

Base class every agent must implement:

- Abstract method: `async handle(task: Task) → TaskResult`
- Lifecycle: `register()`, `deregister()`, `send_heartbeat()`
- Helper: `make_agent_server(agent)` returns FastAPI app with `/run`, `/health`, `/info` endpoints

4. API Contracts

4.1 Task Submission (REST)

```
POST /tasks
Content-Type: application/json
Authorization: Bearer <JWT_TOKEN>

{
  "message": "Write a unit test for the login function",
  "session_id": "user-session-123",
  "agent_id": null,
  "context": {"file": "auth.py", "framework": "pytest"},
  "metadata": {"capability": "testing"}
}

Response (200):
{
  "task_id": "abc123",
  "agent_id": "testing-agent-01",
  "status": "completed",
  "content": "Here's a unit test...",
  "tokens_used": 340,
  "cost_usd": 0.000051,
  "duration_ms": 1240,
  "error": null
}
```

4.2 WebSocket Streaming

```
WebSocket: /ws/{session_id}?token=<JWT_TOKEN>

Client sends:
{"message": "Analyze this bug report", "agent_id": null, "context": {...}}

Server streams:
{"type": "start"}
{"type": "result", "task_id": "...", "agent_id": "...", "content": "..."}
{"type": "done"}
```

4.3 Task Chaining

```
POST /tasks/chain
[
  {"message": "Identify security issues", "metadata": {"capability": "code_review"}},
  {"message": "Based on the issues found, write a fix", "metadata": {"capability": "code_generation"}}
]

Response: [TaskResult1, TaskResult2]
(output of Task1 becomes context for Task2)
```

4.4 Agent Management

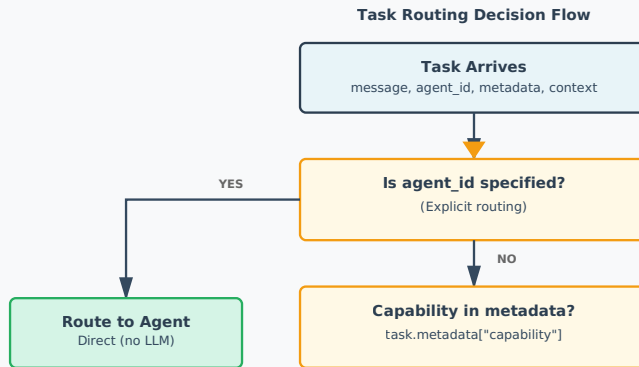
Endpoint	Method	Purpose
<code>/agents/register</code>	POST	Agent joins the platform
<code>/agents/{agent_id}</code>	GET	Get agent details
<code>/agents</code>	GET	List agents (with filters)
<code>/agents/{agent_id}/heartbeat</code>	POST	Keep-alive ping
<code>/agents/{agent_id}</code>	DELETE	Deregister agent

4.5 Analytics & Monitoring

Endpoint	Purpose
<code>GET /health</code>	Platform health + online agents
<code>GET /health/ready</code>	Kubernetes readiness probe
<code>GET /tasks?session_id=...&agent_id=...&limit=50</code>	Task history
<code>GET /usage/summary?agent_id=...</code>	Usage stats (tasks, tokens, cost)
<code>GET /usage/per-agent</code>	Per-agent usage breakdown

5. Task Routing Flow

Figure 2: Task Routing Decision Tree



6. Task Execution Lifecycle

Figure 3: Agent Lifecycle State Machine

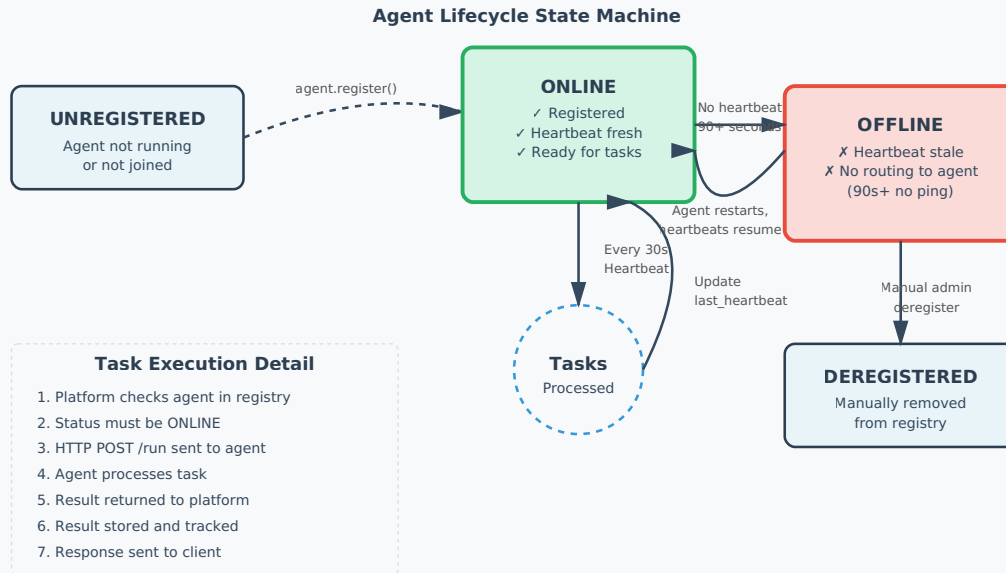
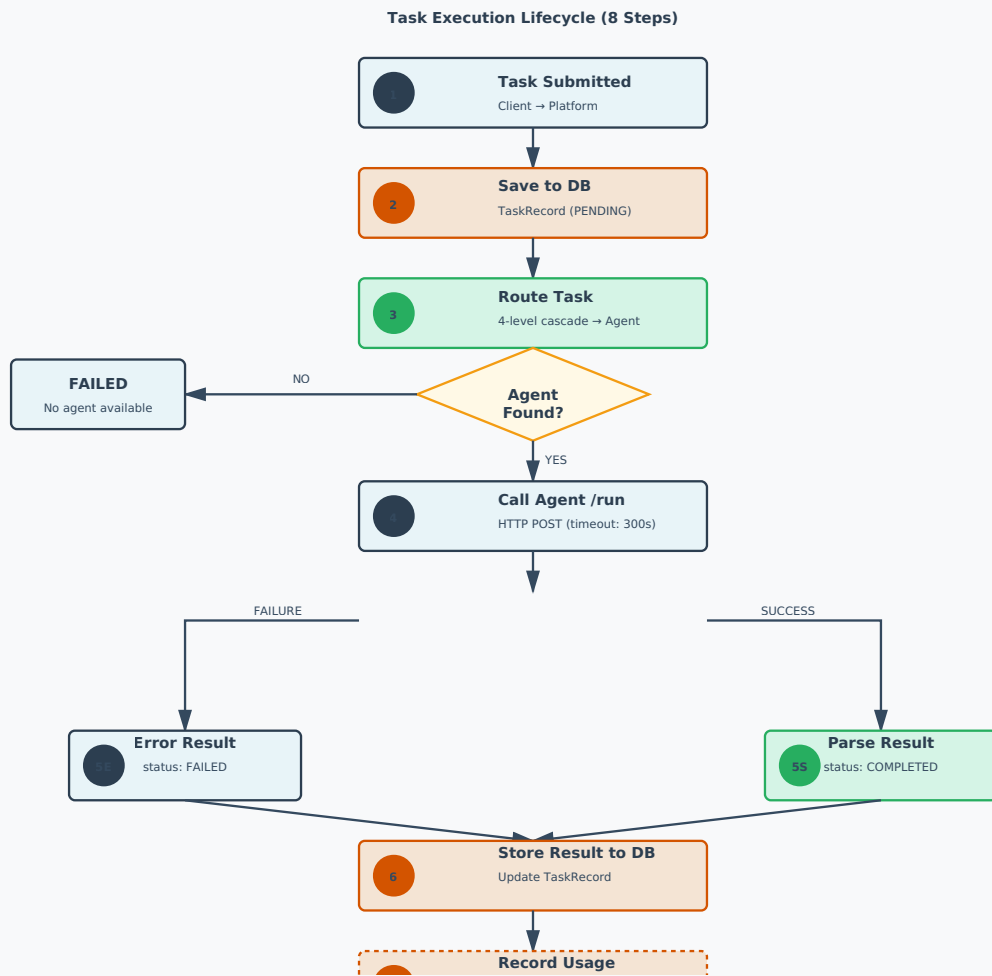


Figure 4: Task Execution Lifecycle (8 Steps)



7. Architecture Diagrams

7.1 Architecture Overview

The diagrams above illustrate the complete architecture of the Omi Platform:

- **Figure 1** shows how clients connect to the platform and how the platform coordinates with agents
- **Figure 2** demonstrates the intelligent 4-level routing cascade that determines which agent handles each task
- **Figure 3** illustrates the agent lifecycle, from registration through active service to eventual deregistration
- **Figure 4** breaks down the 8-step task execution process from submission to completion

7.2 Key Design Patterns

Task Chain Execution

When a client submits multiple tasks via `POST /tasks/chain`, the platform executes them sequentially:

1. Execute Task 1 → Get Result 1
2. Extract result content and store as `prior_result`
3. Pass `prior_result` in the context of Task 2
4. Execute Task 2 (which may use `prior_result`) → Get Result 2
5. Repeat for Task 3, Task 4, etc.
6. If any task fails → stop chain and return partial results

WebSocket Streaming

Clients can stream tasks and results in real-time via WebSocket:

1. Client opens WebSocket: `/ws/SESSION_ID?token=JWT_TOKEN`
2. Platform validates JWT token from query params
3. Client sends task JSON: `{"message": "...", "agent_id": null, ...}`
4. Platform broadcasts events:
 - `{"type": "start"}` — Task processing begun
 - `{"type": "result", ...}` — Task completed with result
 - `{"type": "done"}` — Result finalized
5. Client can send more tasks or close connection

Multi-Agent Collaboration

Send the same task to multiple agents and synthesize their outputs:

1. `POST /tasks/collaborate` with message and `agent_ids: ["agent-1", "agent-2", ...]`
2. Platform submits the same message to all specified agents in parallel
3. Collects all results
4. Joins them with separator: `**[agent-1]**\n...\n\n---\n\n**[agent-2]**\n...`
5. Returns combined result to client

8. Security & Compliance

8.1 Authentication & Authorization

- **JWT tokens** issued at `/auth/token` after credential validation
- **Token scope:** User identity (sub) and role
- **Enforcement:** Middleware checks Bearer token on protected routes
- **Expiration:** 24 hours (configurable)
- **Production:** Replace dev auth with LDAP, OAuth2, or SAML

8.2 Network Security

- **CORS:** Restricted to dashboard URL and localhost (dev)
- **Trusted Hosts:** Only allow requests from whitelisted hosts
- **Security Headers:** HSTS, X-Content-Type-Options, X-Frame-Options, X-XSS-Protection
- **WebSocket Auth:** Token must be provided in query params

8.3 Input Validation

- **Message size limit:** 10 KB per task message
- **WebSocket payload:** 100 KB limit
- **Credential length:** 255 chars max
- **Agent ID format:** UUID or alphanumeric (validated on registration)

8.4 Resource Limits

- **Task timeout:** 300 seconds (5 minutes)
- **Heartbeat timeout:** 90 seconds (agents mark OFFLINE after no ping)
- **Max concurrent requests:** Dependent on FastAPI worker pool
- **Database connections:** Connection pooling (SQLAlchemy)

9. Scaling & Deployment Considerations

9.1 Single Instance (Current)

- ☐ Suitable for < 10 agents, < 1000 tasks/day
- ☐ SQLite database on local disk
- ☐ Minimal operational overhead
- ⚠ Not HA; no failover

9.2 Multi-Instance (Future Path)

Requirements for horizontal scaling:

1. Shared database — Migrate from SQLite to PostgreSQL
2. Distributed registry — Use Redis for agent heartbeat cache
3. Message queue — Kafka/RabbitMQ for async task queuing
4. Load balancer — Nginx/HAProxy to distribute requests
5. Session affinity — Optional; WebSocket sessions can move across instances

9.3 Observability for Operations

- **Metrics:** Prometheus-compatible endpoint for Grafana
- **Logs:** Structured JSON logs to stdout (for aggregation)

- **Traces:** OpenTelemetry instrumentation (optional)
- **Dashboards:** Show online agents, task success rate, cost per agent

10. Extension Points

10.1 Custom Routing Strategy

Override `route()` in `dispatcher/router.py` to implement:

- ML-based agent selection (ranking model)
- Load-based routing (prefer less busy agents)
- Latency-based routing (prefer fast agents)
- Regional routing (geo-distributed agents)

10.2 Custom Storage Backend

Replace SQLAlchemy engine with:

- MongoDB (document storage)
- Cassandra (time-series data)
- DynamoDB (serverless)

10.3 Custom Authentication

Replace JWT with:

- OAuth2 + OpenID Connect
- SAML 2.0
- Kerberos/Active Directory

10.4 Agent Middleware

Wrap `make_agent_server()` to add:

- Rate limiting per user/role
- Request/response logging
- Custom metrics collection
- Request tracing

11. Key Metrics & SLOs

Metric	Target	Notes
Task Success Rate	> 99%	Includes timeouts, agent failures
Agent Availability	> 99.5%	Uptime of registered agents
Task Latency (P99)	< 5 sec	Excluding agent execution time
Routing Latency (P99)	< 100 ms	Time to pick agent (incl. LLM calls)
Platform Uptime	> 99.9%	Exclude infra failures
Cost per Task	< \$0.01 USD	Token cost + platform overhead

Omi Platform — Technical Design Document

Version 0.1.0 | June 2026 | Status: Active Development

With Professional SVG Diagrams

Generated: 2026-06-08